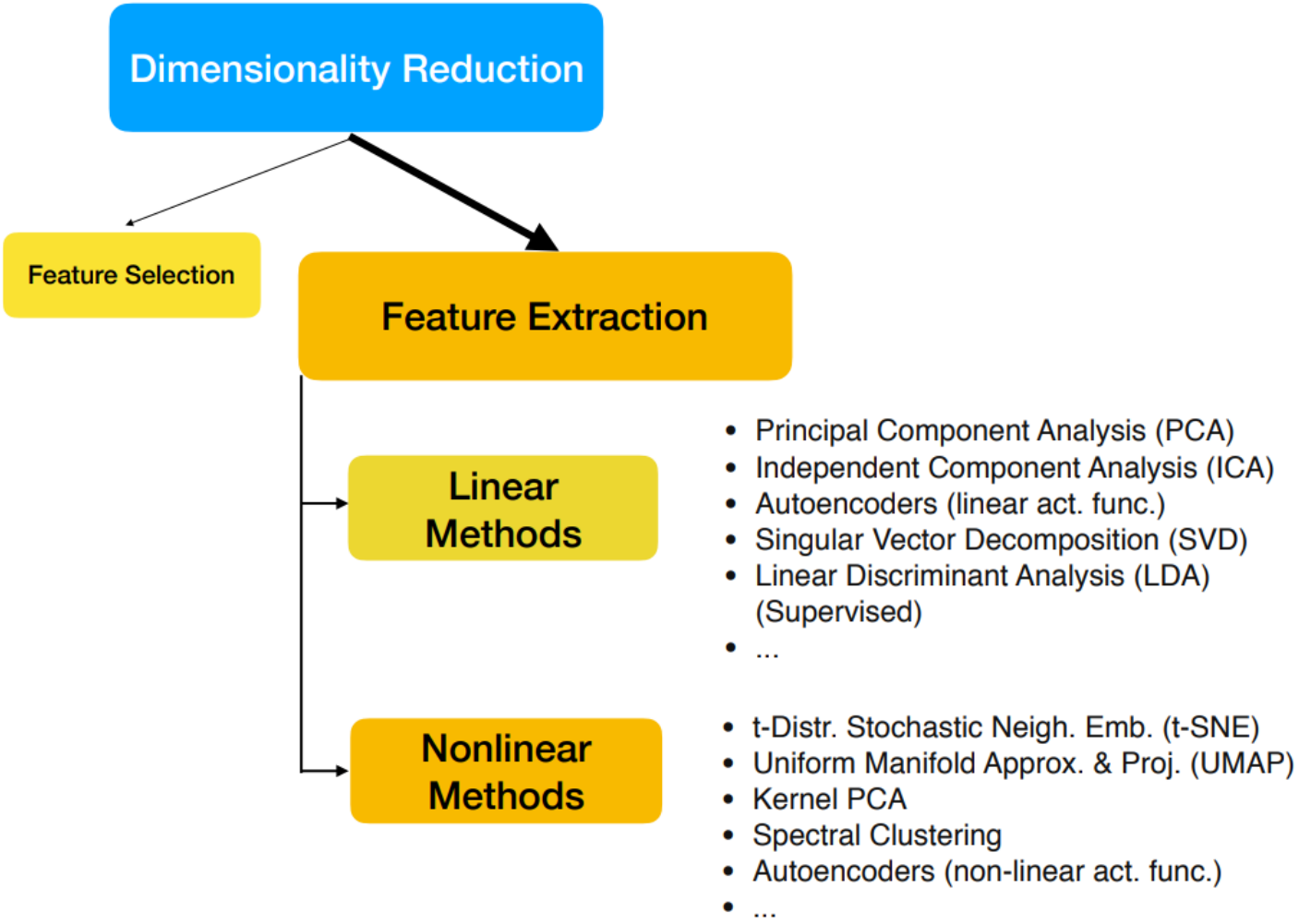
MA 5755: Data Analysis & Visualization in R/Python/SQL

Week 7: Beyond PCA: t-SNE and UMAP

Dr. Rakhi Singh
Department of Mathematics
IIT Madras

Spring 2026

Big picture



Source: `https://sebastianraschka.com/pdf/lecture-notes/stat479fs18/14_feat-extract_slides.pdf`

From PCA to Manifold Learning

We assume a dataset

$$X = \{x_1, \ldots, x_n\}, \qquad x_i \in \mathbb{R}^D$$

Goal: obtain a low-dimensional representation

$$y_i \in \mathbb{R}^d, \qquad d \ll D$$

Previously:

- PCA: linear projection
- Kernel PCA: nonlinear feature map

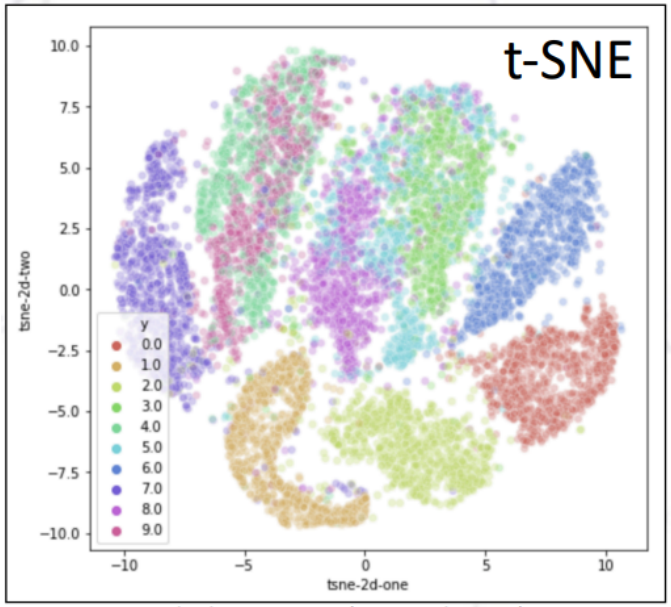
New idea:

- preserve local geometry
- manifold learning

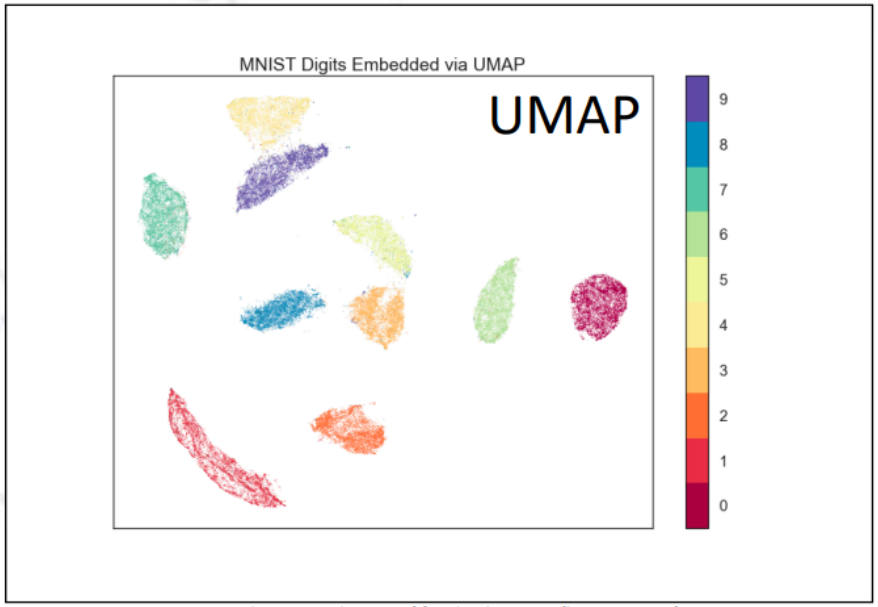
Illustrative explanation

PCA and k-means has been very popular since they can handle the linear case well, but two new(ish) non-linear and powerful candidates are around: t-SNE and UMAP.

Their performance on the MNIST data set.



Source: Towards data science (PCA and t-SNE)



Source: UMAP GitHub page: https://github.com/lmcinnes/umap

t-Distributed Stochastic Neighbor Embedding

t-Distributed Stochastic Neighbor Embedding (t-SNE) constructs a low-dimensional embedding by minimizing the divergence between two probability distributions:

$$KL(P\|Q) = \sum_{i,j} p_{ij} \log \frac{p_{ij}}{q_{ij}},$$

- p_{ij} measures pairwise similarity in the original high-dimensional space
- q_{ij} measures similarity in the low-dimensional embedding

Key advantages:

- Nonlinear dim. reduction capable of capturing complex manifold structure
- No strong assumptions about cluster shapes, distributions, or number of clusters
- Preserves local neighborhood relationships
- Produces clear cluster visualizations in many high-dim datasets

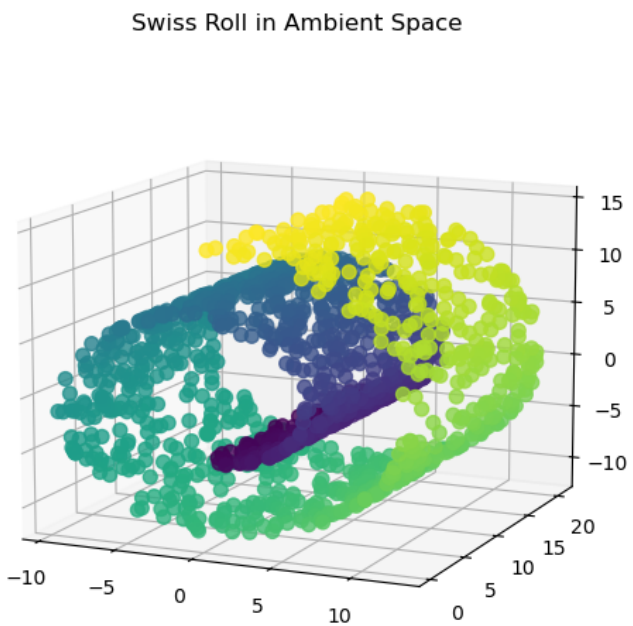
Manifold Hypothesis

Data lies near a low-dimensional manifold $\mathcal{M} \subset \mathbb{R}^D$ with intrinsic dimension $\dim(\mathcal{M}) = d \ll D$.

We seek embedding

$$f : \mathcal{M} \to \mathbb{R}^d$$

that preserves local neighborhood relationships.



What is t-SNE?

Goal: Represent high-dimensional data in a low-dimensional space (typically 2D or 3D) while preserving neighborhood relationships.

Step 1: Compute pairwise similarities in high-dimensional space

- For each pair of points, measure how similar they are.
- Similarity is computed using a distance-based kernel (typically Gaussian).
- Nearby points obtain high similarity scores.
- Result: a probability distribution describing neighborhood relationships in the original space.

Step 2: Define similarities in the low-dimensional embedding

- Points are placed in a low-dimensional map.
- Similarities between points are computed using a heavy-tailed distribution (Student-t).
- This helps avoid the *crowding problem*.

What is t-SNE? (contd.)

Step 3: Match the two similarity structures

- The algorithm adjusts the positions of points in the embedding.
- Objective: make the low-dimensional similarities match the high-dimensional similarities.
- This is done by minimizing a divergence between the two distributions using gradient descent.

The algorithm relies on several definitions:

- Pairwise similarity between points using a Gaussian kernel
- Conditional probabilities describing neighborhood relationships
- Perplexity parameter controlling effective neighborhood size
- Divergence measure (e.g., **KL divergence**) to match the two distributions

Pairwise Similarity via Gaussian Kernel

Given data points

$$x_1, \ldots, x_n \in \mathbb{R}^D$$

we measure how similar two points are using a Gaussian kernel.

$$p_{j|i} = \frac{\exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma_i^2}\right)}{\sum_{k \neq i} \exp\left(-\frac{\|x_i - x_k\|^2}{2\sigma_i^2}\right)}$$

Interpretation:

- Similarity decreases with squared Euclidean distance
- σ_i controls the local neighborhood width
- Nearby points obtain larger probability

Interpretation of Gaussian Similarity

The Gaussian kernel

$$K(x_i, x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma_i^2}\right)$$

assigns higher similarity to closer points.

$$\|x_i - x_j\| \ll \sigma_i \quad \Rightarrow \quad K(x_i, x_j) \approx 1$$

$$\|x_i - x_j\| \gg \sigma_i \quad \Rightarrow \quad K(x_i, x_j) \approx 0$$

Thus each point defines a **local neighborhood structure**.

Conditional Probabilities of Neighbors

From Gaussian similarities we obtain conditional probabilities

$$p_{j|i}$$

interpreted as

$$P(x_j \text{ is neighbor of } x_i)$$

Properties:

$$p_{j|i} \geq 0, \quad \sum_{j \neq i} p_{j|i} = 1$$

Thus each point defines a probability distribution over its neighbors.

Symmetric Joint Probabilities

t-SNE symmetrizes the neighborhood probabilities.

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$$

Properties:

$$p_{ij} = p_{ji}, \qquad \sum_{i \neq j} p_{ij} = 1$$

The matrix

$$P = (p_{ij})$$

represents the similarity structure of the original data.

Low-Dimensional Similarity

We construct points $y_1, \ldots, y_n \in \mathbb{R}^d$. Similarity in embedding space

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l}(1 + \|y_k - y_l\|^2)^{-1}}$$

This corresponds to a Student-t distribution with one df.

Gaussian kernels (if used in high-dimensional) cause the *crowding problem*.

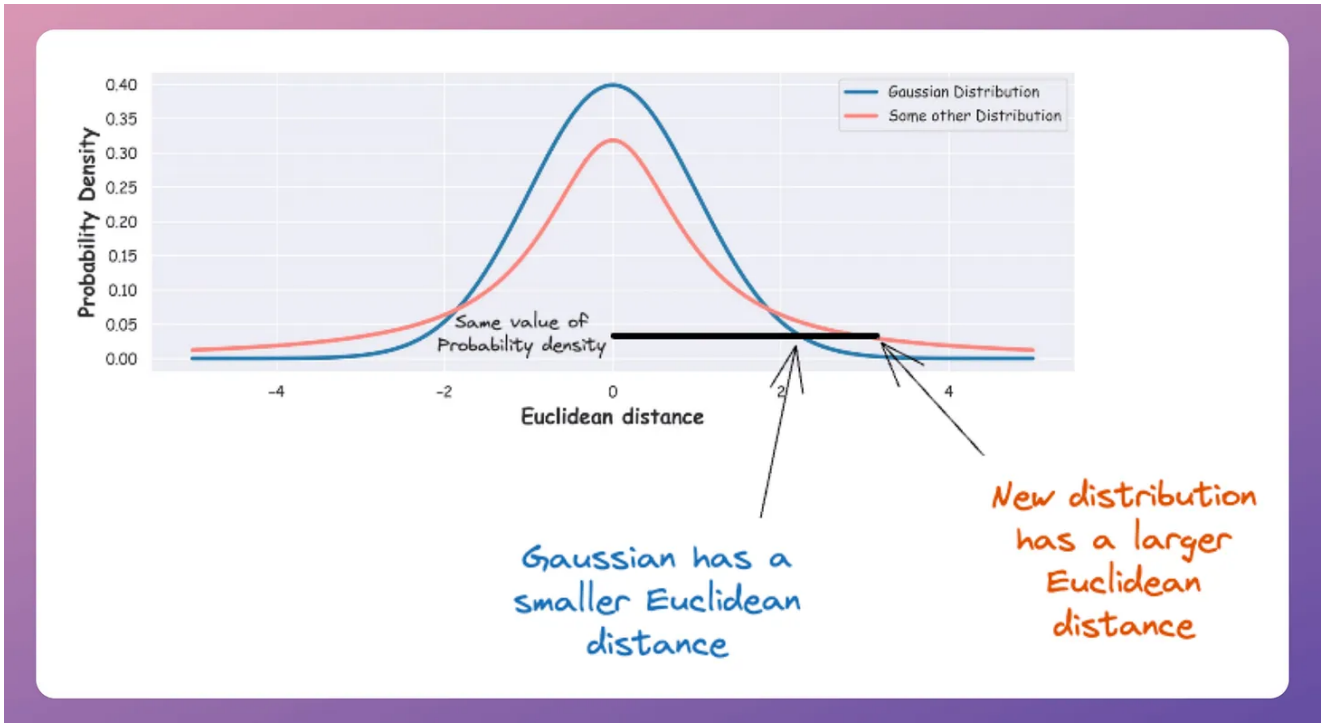
Student-t distribution has heavier tails.

$$t(x) \propto (1 + x^2)^{-1}$$

Effects: nearby points remain close, and distant points repel each other.

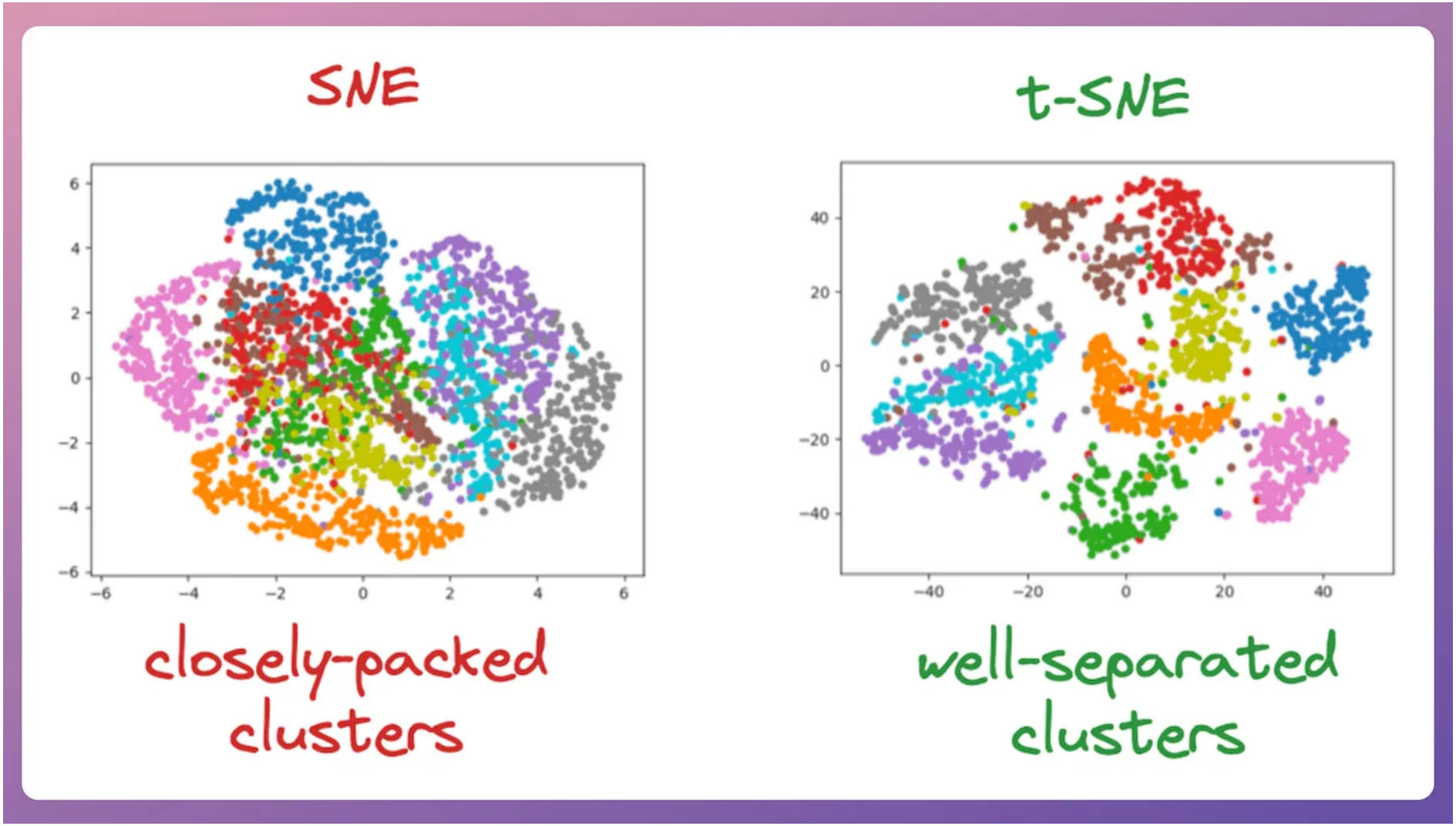
Illustration to see why t-dist

We want to use some other probability distribution, such that, for distant points, we get the *same value of the conditional probability* as we would have obtained from a Gaussian distribution, but at a *larger Euclidean distance*.



Source: `https://blog.dailydoseofds.com/p/t-sne-vs-sne-whats-the-difference`

Crowding Problem (avoided)



Source: `https://blog.dailydoseofds.com/p/t-sne-vs-sne-whats-the-difference`

Matching Two Probability Distributions: KL Divergence

Let $P = (p_{ij})$ be the similarity distribution in high dimension.

For embedding points $y_1, \ldots, y_n \in \mathbb{R}^d$, the corresponding distribution is $Q = (q_{ij})$.

The embedding is obtained by minimizing

$$KL(P \| Q) = \sum_{i,j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

Interpretation:

- if p_{ij} is large, y_i and y_j should be close
- if p_{ij} is small, they may be far apart

Optimization is performed using gradient descent (gradients found with respect to y_i).

Perplexity and Neighborhood Size

The bandwidth σ_i is determined by the perplexity parameter.
Entropy of neighborhood distribution:

$$H(P_i) = -\sum_j p_{j|i} \log p_{j|i}$$

Perplexity is defined as

$$Perp(P_i) = 2^{H(P_i)}$$

Small perplexity focuses on very local structure and emphasizes small clusters, whereas large perplexity considers larger neighborhoods and preserves broader structure.

Effective number of neighbors typical values: 5–50

t-SNE: Limitations

- **Computational cost**: Pairwise similarities must be computed for all points, giving complexity $O(N^2)$, which becomes expensive for large datasets.
- **Memory requirements**: The similarity matrix grows quadratically with the number of samples.
- **Weak global structure preservation**: Distances between clusters may not correspond to meaningful global relationships.
- **Hyperparameter sensitivity**: The embedding depends strongly on parameters such as *perplexity* and the learning rate.

Typical applications In real life, the t-SNE algorithm has especially had its impact in DNA research, where the number of cases is typically not that large

Practical Points When Using t-SNE

Interpreting clusters

- t-SNE reveals local clusters effectively.
- Distances between clusters may **not be meaningful**.

Axes have no meaning

- The coordinate axes of a t-SNE plot are arbitrary.
- t-SNE is mainly a **visualization tool**, not a predictive model.

Random initialization

- t-SNE starts from a random embedding.
- Results can vary; fix a **random seed** for reproducibility.

Preprocessing and dimensionality

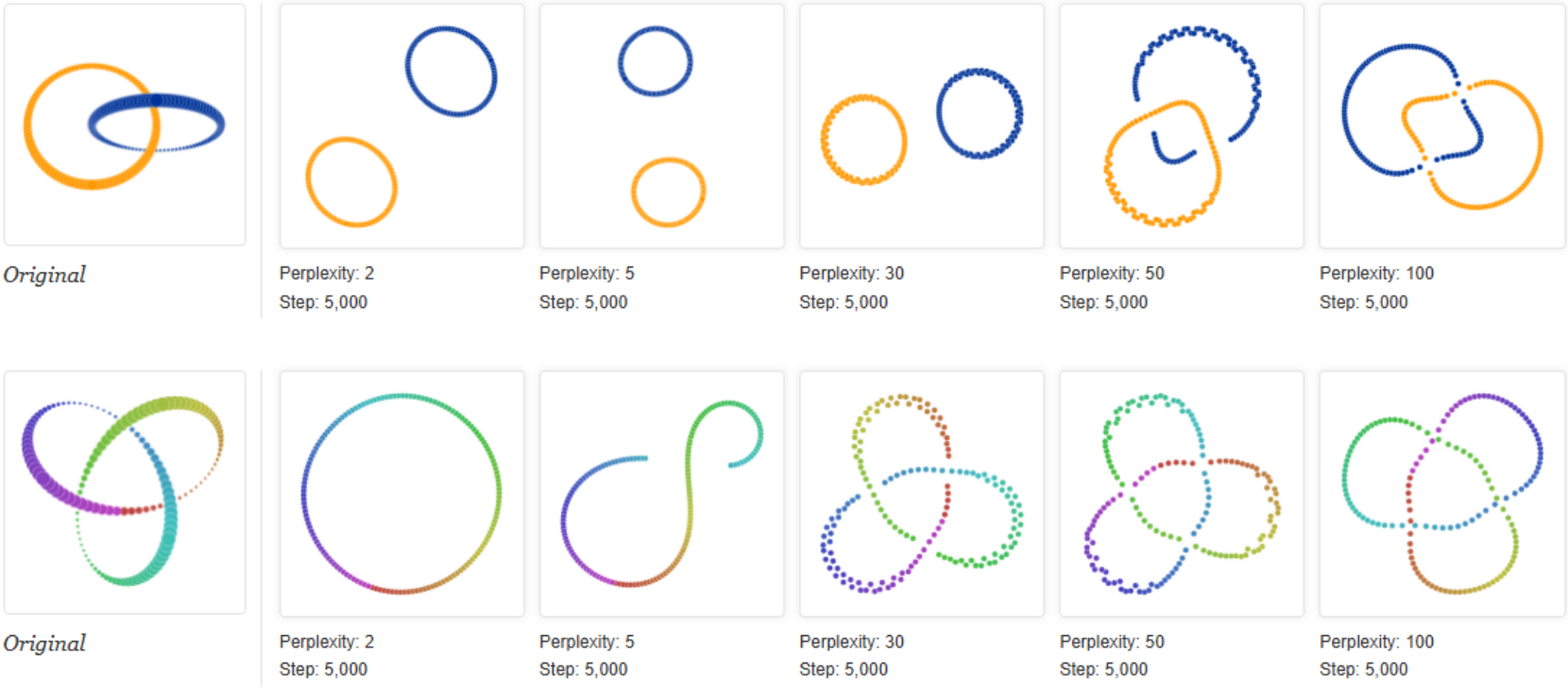
- Scaling or normalizing features can affect the embedding. **PCA first**?

Other parameters

- Learning rate and number of iterations influence convergence.
- t-SNE is powerful but not universal.

t-SNE Example

Should look at a wide range of parameters to understand geometry.



Source: https://distill.pub/2016/misread-tsne/

Also, look at the same link for a nice visual playground along with many more examples!

Outline

1 Uniform Manifold Approximation and Projection

Uniform Manifold Approximation and Projection (UMAP)

UMAP (a nonlinear dimensionality reduction method): based on manifold learning and topological structure.

Instead of directly matching pairwise probabilities as in t-SNE, UMAP:

- Constructs a weighted graph describing local neighborhoods in the high-dimensional space
- Interprets this graph using concepts from topological data analysis
- Finds a low-dimensional embedding whose graph structure is as similar as possible

Key advantages:

- Often preserves both **local and global structure**
- Typically **much faster** than t-SNE on large datasets
- Provides a mathematically grounded framework via **Riemannian geometry and topology**

UMAP: Conceptual Idea

Assume data lies on a manifold

$$\mathcal{M} \subset \mathbb{R}^D$$

with intrinsic dimension $d \ll D$.

UMAP attempts to:

1. Approximate the manifold structure using a **nearest-neighbor graph**
2. Interpret this graph as a **fuzzy topological representation**
3. Construct a low-dimensional embedding whose fuzzy topology is similar

Thus UMAP transforms the problem of dimensionality reduction into:

matching two fuzzy topological structures.

Step 1: Construct Local Neighborhood Graph

Given data $x_1, \ldots, x_n \in \mathbb{R}^D$, UMAP first identifies the k nearest neighbors of each point.

Edges between neighboring points are assigned weights representing similarity.

Similarity is defined as

$$w_{ij} = \exp\left(-\frac{\max(0, d(x_i, x_j) - \rho_i)}{\sigma_i}\right)$$

where

- $d(x_i, x_j)$ is the distance between points
- ρ_i ensures at least one close neighbor
- σ_i adjusts local scaling

Result: a weighted **k-nearest neighbor graph** capturing local geometry.

Fuzzy Simplicial Set Representation

UMAP interprets the neighborhood graph using ideas from topology.

Each edge weight w_{ij} represents the strength of membership of the edge between x_i and x_j.

This produces a structure known as a

fuzzy simplicial set

which encodes neighborhood relationships of the data manifold.

Key intuition:

- Strong edges represent highly likely neighbors
- Weak edges represent uncertain or distant relationships

This representation approximates the topology of the underlying manifold.

Step 2: Low-Dimensional Representation

We now place points

$$y_1, \ldots, y_n \in \mathbb{R}^d$$

in a low-dimensional space.

UMAP defines similarity in the embedding using a smooth curve:

$$q_{ij} = \frac{1}{1 + a\|y_i - y_j\|^{2b}}$$

where parameters a, b control the shape of the curve.

This function behaves similarly to a heavy-tailed distribution:

- nearby points attract strongly
- distant points exert weak influence

Step 3: Matching the Two Structures

Let

- w_{ij} represent high-dimensional neighborhood strengths
- q_{ij} represent similarities in the embedding

UMAP finds the embedding by minimizing the cross-entropy:

$$\mathcal{L} = \sum_{i,j} \left[w_{ij} \log \frac{w_{ij}}{q_{ij}} + (1 - w_{ij}) \log \frac{1 - w_{ij}}{1 - q_{ij}} \right]$$

Interpretation:

- Strong edges in the original graph should remain close in the embedding
- Weak or absent edges should remain far apart

Optimization is performed using stochastic gradient descent.

Important UMAP Parameters

Number of neighbors ($n_{\text{neighbors}}$)

- Controls the size of the local neighborhood graph
- Small values emphasize local clusters
- Larger values preserve more global structure

Minimum distance (*min_dist*)

- Controls how tightly points cluster in the embedding
- Small values produce compact clusters
- Larger values spread clusters apart

These parameters determine the balance between local and global structure.

Practical Advantages of UMAP

Compared with t-SNE:

- **Scales to large datasets** (millions of points)
- Often **preserves more global relationships**
- Provides a **faster optimization procedure**
- Produces embeddings suitable for downstream tasks

UMAP is widely used in:

- single-cell genomics
- image embeddings
- natural language representations
- exploratory data analysis

Why UMAP Scales Better Than t-SNE

The computational structure of UMAP differs significantly from t-SNE.

t-SNE

- Requires computing similarities between many pairs of points.
- Similarity matrix grows roughly as $O(N^2)$.
- Expensive for large datasets.

UMAP

- Uses a **k-nearest neighbor graph** instead of all pairwise similarities.
- Graph is sparse: each point connects only to nearby neighbors.
- Optimization operates on this sparse structure.

Result:

$$\text{UMAP complexity is closer to } O(N \log N)$$

which allows embeddings of much larger datasets.

UMAP vs t-SNE

	t-SNE	UMAP
Idea	Prob. matching	Topological graph
Structure	Local	Local + global
Objective	KL div.	Cross-entropy
Scaling	Expensive for large N	Scales better
Embedding Stability	Sensitive	More stable

Despite the differences, both methods are powerful visualization tools for high-dimensional data.